Recurring Investments Feature - Complete Documentation

Overview

This document describes the comprehensive recurring investments (SIP/RD) feature implementation in the FinPlanner application. The feature enables users to set up, manage, and track systematic investment plans (SIPs), recurring deposits (RDs), and other recurring investments with automatic bill generation and portfolio tracking.

Feature Requirements Met

✅ All 8 feature requirements have been successfully implemented:

1. **Investments Tab - Recurring Investment Creation** ✓
2. **Investments Tab - Recurring SIPs Table** ✓
3. **Auto-Generate Bills** ✓
4. **Bill Payment Integration** ✓
5. **Dashboard - Upcoming SIPs Metric** ✓
6. **Bills Tab - SIP Bill Filter** ✓
7. **Demo Data & Testing** ✓
8. **Documentation** ✓

---

1. Architecture & Implementation

Database Schema

The feature utilizes existing Prisma models with NO schema changes required:

- **Investment Model**: Stores investment details including quantity, prices, and values
- **SIP Model**: Tracks recurring investment schedules and installments
- **Bill Model**: Links to investments via `linkedInvestmentId` field (already existed)
- **BillInstance Model**: Tracks individual bill payments

Key Relationships

```
Bill {
  linkedInvestmentId String?
  linkedInvestment   Investment?
}

SIP {
  investmentId String
  investment   Investment
}
```

---

2. Components Created

2.1 Add Recurring Investment Dialog

File: `/app/components/add-recurring-investment-dialog.tsx`

Features:
- Instrument name and symbol/code input
- Investment type selection (Mutual Fund SIP, ELSS, RD, PPF, etc.)
- Platform selection (Groww, Zerodha, Paytm Money, etc.)
- Amount per installment (INR)
- Frequency selection (Monthly, Quarterly, Yearly)
- Unit price/NAV input with auto-calculation of units
- Start date picker
- Goal and category linking (optional)
- Description field

Workflow:
1. User fills in investment details
2. Units are auto-calculated: `units = amount / unitPrice`
3. Current value is initially set to amount invested
4. On submission:
- Creates Investment record
- Creates SIP record
- Auto-generates Bill record (via SIP API)
5. Shows success toast with bill creation notification

Styling: Matches existing shadcn/ui dialog patterns with emerald color scheme for SIP-related actions.

2.2 Recurring Investments Table

File: `/app/components/recurring-investments-table.tsx`

Features:
- **Summary Cards** (4 metrics):
- Total SIPs count
- Active SIPs count
- Monthly Investment amount (sum of all monthly SIPs)
- Total Portfolio Value (sum of all SIP investment values)

- **Data Table** with columns:
- Instrument Name (with installments paid count)
- Type (asset class badge)
- Amount (per installment)
- Frequency
- Units (3 decimal precision)
- Current Value
- Next Due Date
- Status (with color-coded badges)
- Actions (Pause/Resume, Delete)

Status Management:
- Active → Paused: Toggles SIP status

- Paused → Active: Resumes SIP
- Delete: Removes SIP (keeps investment intact)

Empty State: Shows friendly message when no SIPs exist.

Styling: Professional card-based layout matching existing investment tables.

2.3 Upcoming SIPs Card (Dashboard)

File: `/app/components/upcoming-sips-card.tsx`

Features:
- Displays count of SIP bills due in next 30 days
- Shows total amount due
- Links to Bills tab with SIP filter applied
- Emerald color scheme for consistency

Styling: Matches existing dashboard metric cards with hover effects.

---

3. API Endpoints

3.1 Enhanced SIP Creation

File: `/app/app/api/investments/sips/route.ts`

Endpoints:
- `POST /api/investments/sips` - Create new SIP

Enhancement:
- Auto-creates a Bill record when SIP frequency is MONTHLY
- Finds or creates “Investment” category
- Links bill to investment via `linkedInvestmentId`
- Sets bill frequency to match SIP frequency
- Uses SIP start date as bill’s first due date

3.2 SIP Management

File: `/app/app/api/investments/sips/[id]/route.ts`

Endpoints:
- `PATCH /api/investments/sips/[id]` - Update SIP status/details
- Used for Pause/Resume functionality
- Updates status, amount, frequency, endDate

- `DELETE /api/investments/sips/[id]` - Delete SIP
- Removes SIP record
- Investment record remains intact

3.3 Enhanced Bill Payment

File: `/app/app/api/bills/[id]/payment/route.ts`

Enhancement - Investment Update Logic:
When a bill linked to an investment is marked as paid:

1. **Calculate Units to Add**:
   ```
   javascript
     if (investment.currentPrice > 0) {
       unitsToAdd = paymentAmount / investment.currentPrice
     } else if (investment.averagePrice > 0) {
       unitsToAdd = paymentAmount / investment.averagePrice
     } else {
       unitsToAdd = paymentAmount // For FD, RD (1:1 ratio)
     }
   ```

2. **Update Investment**:
   - Increment `currentValue` by payment amount
   - Increment `totalInvested` by payment amount
   - Increment `quantity` by calculated units
   - Recalculate `averagePrice`

3. **Create Transaction** (if enabled)

4. **Roll Next Due Date** (for recurring bills)

3.4 Dashboard Enhancement

File: `/app/app/api/dashboard/route.ts`

Addition:
- Queries bills with `linkedInvestmentId` not null
- Filters by next 30 days
- Returns count and total amount
- Added to dashboard response:

```
javascript
  upcomingSips: {
    count: number,
    totalAmount: number
  }
```

---

4. Page Integrations

4.1 Investments Page

File: `/app/app/investments/page.tsx`

Changes:
- Added “Start SIP” button (emerald colored)
- Integrated `AddRecurringInvestmentDialog`
- Added `RecurringInvestmentsTable`
- Implemented two-tab layout:
- **All Investments**: Shows regular investments
- **Recurring SIPs**: Shows SIPs table

- Fetches SIPs data from API
- Refreshes both tabs on data change

4.2 Dashboard Page

File: `/app/app/dashboard/page.tsx`

Changes:
- Added `UpcomingSipsCard` import
- Extended `DashboardData` interface with `upcomingSips`
- Added 5th metric card in top row (expanding from 4 to 5 cards)
- Card positioned after “Savings Rate”
- Links to Bills tab with SIP filter

4.3 Bills Page

File: `/app/app/bills/page.tsx`

Changes:
- Added `filterSIP` state
- Added “SIP Bills Only” toggle button (emerald colored)
- Implemented filter logic: `bills.filter(bill => bill.linkedInvestmentId)`
- Applied filter to:
- Total Bills count
- Paid Bills count
- Pending Bills count
- All bill lists (Overview, Monthly, Yearly views)
- URL parameter support: `?filter=sip`
- Toggle button text changes: “SIP Bills Only” ↔ “All Bills”

---

5. Demo Data

5.1 Seed Script Updates

File: `/app/prisma/seed.ts`

Added Demo Data:

1. **5 Recurring Investments**:
   - SBI Bluechip Fund (Mutual Fund, Monthly, ₹5,000)
   - ICICI Prudential Equity Fund (ELSS, Monthly, ₹10,000)
   - HDFC Bank RD (Recurring Deposit, Monthly, ₹1,000)
   - Axis Long Term Equity Fund (ELSS, Quarterly, ₹20,000)
   - Kotak Equity Opportunities Fund (Mutual Fund, Monthly, ₹7,500)

2. **5 SIP Records**:
   - Different frequencies (Monthly, Quarterly)
   - Varied installments paid (1-4)
   - Next dates spread across Feb-Apr 2026
   - All marked as ACTIVE

3. **5 Auto-Generated Bills**:
   - Linked to respective investments

- Matching amounts and frequencies
- Due dates aligned with SIP next dates
- Category: “Investment”

Running the Seed:

```
cd /home/ubuntu/finplanner/app
npx prisma db seed
```

---

6. User Workflows

6.1 Creating a New SIP

Steps:
1. Navigate to **Investments** tab
2. Click **“Start SIP”** button (emerald, top-right)
3. Fill in the form:
- Instrument Name: “Axis Bluechip Fund”
- Investment Type: “Mutual Fund SIP”
- Platform: “Groww”
- Amount: ₹5000
- Frequency: “Monthly”
- Unit Price: ₹150
- Start Date: Select date
4. (Optional) Link to Goal or Category
5. Click **“Create Recurring Investment”**

Result:
- Investment created with initial units
- SIP record created
- Bill auto-generated
- Success toast: “Recurring investment created successfully!”
- Info toast: “A bill has been auto-generated for this SIP”

6.2 Viewing SIPs

Steps:
1. Navigate to **Investments** tab
2. Click **“Recurring SIPs”** tab
3. View summary metrics at top
4. Browse SIPs table

Information Displayed:
- All SIP details in table format
- Status badges (Active, Paused, etc.)
- Next due dates
- Current portfolio values

6.3 Managing SIPs

Pause a SIP:
1. In Recurring SIPs table
2. Click **Pause** button (outline)
3. Confirmation toast
4. Status updates to “PAUSED”

Resume a SIP:
1. Click **Play** button
2. Status updates to “ACTIVE”

Delete a SIP:
1. Click **Delete** button (red)
2. Confirm deletion in browser prompt
3. SIP removed (investment retained)

6.4 Paying SIP Bills

Steps:
1. Navigate to **Bills** tab
2. (Optional) Click **“SIP Bills Only”** to filter
3. Select a month/year view
4. Find SIP bill (shows “Investment” category)
5. Click **“Mark Paid”** button
6. Enter payment details (amount, reference, notes)
7. Enable “Create Transaction” if needed
8. Click **“Mark as Paid”**

Result:
- Bill instance marked as paid
- Investment updated:
- Units increased
- Current value increased
- Total invested increased
- Average price recalculated
- Transaction created (if enabled)
- Next due date rolled forward

6.5 Viewing Upcoming SIPs (Dashboard)

Steps:
1. Navigate to **Dashboard**
2. View **“Upcoming SIPs”** metric card (top row, 5th position)

Information:
- Count of SIP bills due in next 30 days
- Total amount due
- Click card or “View SIP bills” link → Bills tab with SIP filter

---

7. Styling & Design

Color Scheme

- **SIP Actions**: Emerald (`emerald-600`, `emerald-700`)
- **Active Status**: Green badges
- **Paused Status**: Yellow badges
- **Investment Category**: Blue/purple tones
- **Metric Cards**: Varied colors (blue, emerald, purple, green)

UI Components (shadcn/ui)

- **Dialog**: For SIP creation
- **Table**: For SIPs list
- **Card**: For metrics and containers
- **Badge**: For status indicators
- **Button**: For actions (primary, outline, ghost variants)
- **Input, Select, Label**: For form fields
- **Tabs**: For view switching

Consistency

All components match existing FinPlanner patterns:
- Card-based layouts
- Hover effects with shadows
- Professional color palette
- Responsive grid layouts
- Icon usage from lucide-react
- Tailwind CSS utility classes

---

8. Technical Implementation Details

State Management

Investments Page:

```
const [showAddSIPDialog, setShowAddSIPDialog] = useState(false)
const [data, setData] = useState<InvestmentsData>({
  investments: [],
  sips: [],
  portfolio: {...}
})
```

Bills Page:

```
const [filterSIP, setFilterSIP] = useState(false)
const filteredBills = filterSIP
  ? data.bills.filter(bill => bill.linkedInvestmentId)
  : data.bills
```

Data Fetching

Parallel fetches for performance:

```
const [investmentsRes, portfolioRes, sipsRes] = await Promise.all([
  fetch('/api/investments?includeGoals=true'),
  fetch('/api/investments/portfolio'),
  fetch('/api/investments/sips')
])
```

Auto-Calculation Logic

Units Calculation:

```
useEffect(() => {
  if (formData.amount && formData.unitPrice) {
    const calculatedUnits = parseFloat(formData.amount) / parseFloat(form-
Data.unitPrice)
    setFormData(prev => ({
      ...prev,
      units: calculatedUnits.toFixed(3),
      currentValue: formData.amount
    }))
  }
}, [formData.amount, formData.unitPrice])
```

Error Handling

- API errors shown via toast notifications
- Form validation for required fields
- Loading states for async operations
- Graceful fallbacks for missing data

---

9. Files Modified/Created

Created Files (8)

1. `/app/components/add-recurring-investment-dialog.tsx` - SIP creation dialog
2. `/app/components/recurring-investments-table.tsx` - SIPs table component
3. `/app/components/upcoming-sips-card.tsx` - Dashboard metric card
4. `/app/app/api/investments/sips/[id]/route.ts` - SIP management API
5. `/home/ubuntu/finplanner/RECURRING_INVESTMENTS_FEATURE.md` - This documentation

Modified Files (5)

1. `/app/app/investments/page.tsx` - Added SIPs tab and dialog
2. `/app/app/dashboard/page.tsx` - Added upcoming SIPs card
3. `/app/app/bills/page.tsx` - Added SIP filter
4. `/app/app/api/bills/[id]/payment/route.ts` - Enhanced with investment updates
5. `/app/app/api/dashboard/route.ts` - Added upcoming SIPs data
6. `/app/app/api/investments/sips/route.ts` - Enhanced with auto-bill creation
7. `/app/prisma/seed.ts` - Added demo SIP data

10. Testing Guide

10.1 Setup

```
# Navigate to app directory
cd /home/ubuntu/finplanner/app

# Install dependencies (if not already done)
npm install

# Reset database and seed with demo data
npx prisma db push --force-reset
npx prisma db seed

# Start development server
npm run dev
```

10.2 Test Scenarios

Test 1: View Demo SIPs

Expected: 5 SIPs visible in Recurring SIPs tab
- ✓ All have correct amounts, frequencies
- ✓ Status badges show “ACTIVE”
- ✓ Summary cards show correct totals

Test 2: Create New SIP

1. Click “Start SIP”
2. Fill form with test data
3. Submit

Expected:
- ✓ Success toast appears
- ✓ SIP appears in table
- ✓ Bill created in Bills tab
- ✓ Investment created with correct units

Test 3: Dashboard Metric

1. Navigate to Dashboard
2. View “Upcoming SIPs” card

Expected:
- ✓ Shows count of bills due in next 30 days
- ✓ Shows total amount
- ✓ Link works and applies filter

Test 4: SIP Filter in Bills

1. Navigate to Bills tab
2. Click “SIP Bills Only”

Expected:
- ✓ Only SIP-linked bills visible
- ✓ Stats update correctly

- ✓ Toggle button text changes
- ✓ All views (Overview, Monthly, Yearly) respect filter

Test 5: Mark SIP Bill as Paid

1. Go to Bills → Monthly View
2. Select a month with SIP bill
3. Mark SIP bill as paid with ₹5000

Expected:
- ✓ Bill marked as paid
- ✓ Investment units increase
- ✓ Investment value increases by ₹5000
- ✓ Average price recalculates
- ✓ Next due date rolls forward

Test 6: Pause/Resume SIP

1. Go to Recurring SIPs table
2. Click Pause on an active SIP
3. Verify status changes to PAUSED
4. Click Resume (Play icon)
5. Verify status returns to ACTIVE

Expected:
- ✓ Status toggles correctly
- ✓ Toast notifications appear
- ✓ Changes persist after refresh

Test 7: Delete SIP

1. Click Delete on a SIP
2. Confirm deletion

Expected:
- ✓ SIP removed from table
- ✓ Investment still exists in All Investments tab
- ✓ Linked bill may remain (as per business logic)

---

11. Known Limitations & Future Enhancements

Current Limitations

1. **No Auto-Payment**: Bills must be manually marked as paid
2. **No SIP Editing**: Only status can be updated (amount/frequency changes require new SIP)
3. **No End Date UI**: Total installments field not exposed in creation dialog
4. **No SIP History**: No detailed view of past installments

Potential Future Enhancements

1. **Auto-Payment Integration**:
   - Bank API integration
   - Payment gateway for direct SIP deduction

2. **SIP Modification**:
   - Edit amount/frequency dialog
   - Step-up SIP support (increasing amount over time)

3. **Advanced Analytics**:
   - XIRR calculation
   - Returns comparison charts
   - Performance benchmarking

4. **Notifications**:
   - Email/SMS reminders for due SIPs
   - Payment failure alerts

5. **Bulk Operations**:
   - Pause/Resume all SIPs
   - Export SIP portfolio report

6. **Smart Suggestions**:
   - AI-powered SIP amount recommendations
   - Portfolio rebalancing suggestions

---

12. Troubleshooting

Issue: SIP bills not appearing in Bills tab

Solution: Check if `linkedInvestmentId` is set on the bill. Verify using:

```
SELECT id, name, "linkedInvestmentId" FROM bills WHERE "linkedInvestmentId" IS NOT NULL;
```

Issue: Investment units not updating on payment

Solution: Check API logs for errors. Verify:
- Bill has `linkedInvestmentId`
- Investment `currentPrice` or `averagePrice` > 0
- Payment API completes successfully

Issue: Dashboard metric shows 0 upcoming SIPs

Solution: Verify:
- Bills exist with `linkedInvestmentId`
- Bills have `nextDueDate` within next 30 days
- Bills are `isActive = true`

Issue: Filter button not working in Bills tab

Solution: Check:
- `filteredBills` variable is used instead of `data.bills`
- State updates propagate to all views
- URL parameter parsing works correctly

---

13. API Reference

Get All SIPs

```
GET /api/investments/sips
```

Query Parameters:
- `investmentId` (optional): Filter by investment
- `status` (optional): Filter by status (ACTIVE, PAUSED, etc.)

Response:

```
[
  {
    "id": "cuid",
    "name": "SBI Bluechip Fund - SIP",
    "amount": 5000,
    "frequency": "MONTHLY",
    "status": "ACTIVE",
    "nextDate": "2026-02-05",
    "installmentsPaid": 3,
    "investment": {
      "id": "cuid",
      "name": "SBI Bluechip Fund",
      "assetClass": "MUTUAL_FUNDS"
    }
  }
]
```

Create SIP

```
POST /api/investments/sips
```

Body:

```
{
  "investmentId": "cuid",
  "name": "SBI Bluechip Fund - SIP",
  "amount": 5000,
  "frequency": "MONTHLY",
  "startDate": "2026-01-01",
  "endDate": null,
  "totalInstallments": null
}
```

Auto-creates: Bill record for MONTHLY frequency

Update SIP

```
PATCH /api/investments/sips/[id]
```

Body:

```
{
  "status": "PAUSED",
  "amount": 5000,
  "frequency": "MONTHLY"
}
```

Delete SIP

```
DELETE /api/investments/sips/[id]
```

Response: `{ "success": true, "message": "SIP deleted successfully" }`

Mark Bill as Paid

```
POST /api/bills/[id]/payment
```

Body:

```
{
  "view": "monthly",
  "year": 2026,
  "month": 1,
  "amount": 5000,
  "notes": "Paid via UPI",
  "referenceNumber": "REF123",
  "createTransaction": true
}
```

Effect: Updates linked investment if `linkedInvestmentId` exists

Dashboard Data

```
GET /api/dashboard
```

Response (partial):

```
{
  "upcomingSips": {
    "count": 4,
    "totalAmount": 23500
  },
  ...
}
```

---

14. Conclusion

The recurring investments feature has been successfully implemented with all requirements met. The feature provides:

- ✅ **Comprehensive UI** for SIP management

- ✅ **Automatic bill generation** for recurring investments
- ✅ **Investment tracking** with units and value updates
- ✅ **Dashboard metrics** for quick insights
- ✅ **Filtering capabilities** in Bills tab
- ✅ **Demo data** for immediate testing
- ✅ **Consistent styling** matching existing app design

The implementation is production-ready and follows FinPlanner's existing architecture and design patterns. All components are reusable, maintainable, and extensible for future enhancements.

---

15. Support & Maintenance

Code Locations

- **Components**: `/app/components/`
- **API Routes**: `/app/app/api/`
- **Pages**: `/app/app/`
- **Database**: `/app/prisma/`

Key Developers

This feature was implemented as part of the FinPlanner enhancement project.

Documentation Updates

This documentation should be updated when:
- New features are added
- API endpoints change
- UI components are modified
- Business logic is updated

Last Updated: January 3, 2026

End of Recurring Investments Feature Documentation